

Material instrucional

Geração de Testes Unitários com IA

Atividade prática com Python, pytest e Kiro

Objetivo: analisar regras de negócio, gerar testes unitários com apoio de IA, executar os testes, interpretar falhas e validar cenários positivos, negativos e limites.

Etapa	Objetivo
Demonstração guiada	Criar testes para a função calcular_frete e compreender o papel da IA na geração inicial dos testes.
Regressão controlada	Alterar a função propositalmente, executar os testes novamente e interpretar a falha.
Desafio 1	Validar regras de senha com cenários válidos, inválidos e limites.
Desafio 2	Validar regras de desconto, exceções e prioridade entre condições.
Desafio 3	Validar fluxos condicionais e prioridade na classificação de chamados.

1. Visão geral da prática

Esta atividade trabalha a criação de testes unitários com apoio de IA. O foco é usar a IA como ferramenta de aceleração, sem abrir mão da análise crítica sobre as regras de negócio, os cenários gerados e os resultados obtidos na execução dos testes.

- **Análise de função Python:** identificar entradas, saídas, regras, exceções e comportamentos esperados.
- **Geração de testes com IA:** criar testes unitários usando prompts contextualizados.
- **Execução com pytest:** rodar os testes no terminal e verificar se passam ou falham.
- **Interpretação de falhas:** entender qual regra foi violada quando um teste falha.
- **Validação crítica:** avaliar se os testes gerados realmente cobrem casos relevantes.

2. Pré-requisitos

Antes de iniciar, confirme que o ambiente está preparado para executar código Python e testes automatizados.

- Python instalado no ambiente de desenvolvimento.
- Acesso ao projeto ou pasta local onde os arquivos serão criados.
- Kiro aberto com a pasta do projeto carregada.

- pytest instalado para execução dos testes.
- Conhecimento básico sobre funções, condicionais e exceções em Python.

```
# Comandos básicos
pip install pytest
pytest
```

3. Estrutura inicial do projeto

Crie uma pasta simples para a demonstração guiada. Essa estrutura separa a função principal do arquivo de testes, seguindo a convenção comum do pytest.

```
aula_testes/
|
|-- frete.py
|-- test_frete.py
```

Observação

Arquivos de teste geralmente usam o prefixo test_, pois o pytest identifica automaticamente esses arquivos durante a execução.

4. Demonstração guiada: função calcular_frete

Nesta etapa, a função calcular_frete será usada para demonstrar como transformar regras de negócio em testes automatizados.

```
# frete.py
def calcular_frete(valor_compra, distancia_km):
    if valor_compra < 0 or distancia_km < 0:
        raise ValueError("Valores inválidos")

    if valor_compra >= 200:
        return 0

    if distancia_km <= 10:
        return 10

    return 20
```

4.1 Regras de negócio da função

- Valores negativos para compra ou distância devem gerar ValueError.
- Compras com valor maior ou igual a 200 possuem frete grátis.
- Compras abaixo de 200 com distância até 10 km retornam frete 10.
- Compras abaixo de 200 com distância acima de 10 km retornam frete 20.

4.2 Prompt recomendado para geração dos testes

Descrição

Como o Kiro consegue acessar os arquivos do projeto, não é necessário colar todo o código no prompt. Mesmo assim, é importante informar as regras esperadas para orientar a geração dos testes.

Prompt

Analise a função `calcular_frete` no arquivo `frete.py`.

Regras esperadas:

- compras ≥ 200 possuem frete grátis;
- distância ≤ 10 km retorna frete 10;
- distância > 10 km retorna frete 20;
- valores negativos devem lançar `ValueError`.

Gere testes unitários usando `pytest`.

Inclua cenários positivos, cenários negativos e edge cases.

Use nomes descritivos para os testes.

Crie o arquivo `test_frete.py`.

Não altere a função original.

4.3 Execução dos testes

Terminal

`pytest`

- Se todos os testes passarem, a saída indicará sucesso na execução.
- Se algum teste falhar, leia a mensagem de erro para identificar entrada, resultado obtido e resultado esperado.
- A falha não deve ser ignorada: ela aponta uma possível divergência entre código e regra de negócio.

5. Experimento de regressão

Após gerar e executar os testes, será feita uma alteração propositalmente incorreta na função para demonstrar como testes automatizados detectam regressões.

Versão incorreta para teste de regressão

```
def calcular_frete(valor_compra, distancia_km):  
    if distancia_km <= 10:  
        return 10  
  
    if distancia_km > 10:  
        return 20  
  
    if valor_compra >= 200:  
        return 0
```

Terminal

`pytest`

Objetivo do teste

Verificar se a suíte de testes consegue detectar uma alteração que quebrou a regra de frete grátis para compras com valor maior ou igual a 200.

Por que a falha acontece

A versão incorreta verifica primeiro a distância. Assim, uma compra de 200 ou mais com distância até 10 km retorna 10 antes de chegar à regra de frete grátis. O teste falha porque o comportamento esperado era retornar 0.

5.1 Prompts úteis para analisar a falha

Prompts de análise

Explique por que o teste falhou.

Identifique qual regra de negócio foi quebrada.

Sugira edge cases adicionais para a função calcular_frete.

6. Atividade prática

A atividade será realizada em três desafios. Em cada desafio, analise a função, identifique as regras de negócio e use IA para gerar testes unitários com pytest.

Desafio	Foco da validação
Desafio 1	Validação de senha
Desafio 2	Cálculo de desconto e prioridade de regras
Desafio 3	Classificação de chamados por prioridade

- Criar ou completar o arquivo de teste correspondente ao desafio.
- Gerar testes com apoio de IA, mas revisar criticamente os cenários sugeridos.
- Executar os testes com pytest.
- Corrigir o prompt ou os testes caso a cobertura dos cenários esteja incompleta.
- Não alterar a função original durante a geração dos testes.

7. Desafio 1 - Validação de senha

Este desafio trabalha validação de entrada. A função recebe uma senha e retorna True ou False de acordo com regras mínimas de segurança.

```
# senha.py
def validar_senha(senha):
    if len(senha) < 8:
        return False

    if senha.islower():
        return False

    if senha.isalpha():
        return False

    return True
```

7.1 Regras de negócio

- Senha com menos de 8 caracteres deve ser inválida.
- Senha sem letra maiúscula deve ser inválida.
- Senha sem número deve ser inválida.
- Senha com 8 ou mais caracteres, pelo menos uma letra maiúscula e pelo menos um número deve ser válida.

7.2 Questões para orientar a análise

1. Quais regras tornam a senha inválida?
2. Quais cenários precisam ser protegidos pelos testes?

7.3 Prompt de apoio

```
# Prompt
Analise a função validar_senha no arquivo senha.py.
Gere testes unitários em Python usando pytest.
Crie um arquivo chamado test_senha.py.
```

```
Requisitos:
- criar pelo menos 4 testes;
- incluir cenários válidos e inválidos;
- incluir pelo menos 1 edge case;
- usar nomes descritivos para os testes;
- explicar brevemente quais regras foram testadas.
```

Não altere a função original.

8. Desafio 2 - Cálculo de desconto

Este desafio trabalha regras condicionais com ordem de prioridade. O objetivo é verificar se os descontos são aplicados corretamente e se a função trata entradas inválidas.

```
# desconto.py
def calcular_desconto(valor_compra, cliente_vip):
    if valor_compra < 0:
        raise ValueError("Valor inválido")

    if cliente_vip:
        return valor_compra * 0.85

    if valor_compra >= 300:
        return valor_compra * 0.90

    return valor_compra
```

8.1 Regras de negócio

- Valor de compra menor que 0 deve gerar ValueError.
- Cliente VIP recebe 15% de desconto.
- Compra maior ou igual a R\$ 300 recebe 10% de desconto.
- Demais casos não recebem desconto.
- Cliente VIP com compra maior ou igual a R\$ 300 deve receber o desconto VIP de 15%, pois essa regra tem prioridade.

8.2 Questões para orientar a análise

3. Quais regras definem os descontos aplicados?
4. Existe prioridade entre as regras?
5. Quais cenários devem ser validados pelos testes?

8.3 Prompt de apoio

```
# Prompt
Analise a função calcular_desconto no arquivo desconto.py.
```

Regras esperadas:

- valores negativos devem gerar ValueError;
- clientes VIP possuem prioridade de desconto;
- compras acima de R\$ 300 possuem desconto;
- regras devem respeitar a ordem de prioridade.

Gere testes unitários em Python usando pytest.
Crie um arquivo chamado test_desconto.py.

Requisitos:

- criar cenários positivos e negativos;
- incluir testes de exceção;
- incluir edge cases;
- utilizar nomes descritivos para os testes;
- explicar brevemente quais regras foram validadas.

Não altere a função original.

9. Desafio 3 - Classificação de chamados

Este desafio trabalha fluxos condicionais e prioridade entre regras. A função classifica chamados como prioridade alta, média ou baixa.

```
# chamado.py
def classificar_chamado(tipo_problema, cliente_premium):
    if tipo_problema == "sem_sinal":
        return "alta"

    if tipo_problema == "cobranca" and cliente_premium:
        return "alta"

    if tipo_problema == "cobranca":
        return "media"

    return "baixa"
```

9.1 Regras de negócio

- Problema sem_sinal deve retornar prioridade alta.
- Cobrança para cliente premium deve retornar prioridade alta.
- Cobrança para cliente comum deve retornar prioridade média.
- Demais casos devem retornar prioridade baixa.
- A ordem das regras influencia o resultado final.

9.2 Questão para orientar a análise

6. O código implementa corretamente as regras esperadas?

9.3 Prompt de apoio

Prompt

Analise a função classificar_chamado no arquivo chamado.py.

Regras esperadas:

- problemas de sem_sinal possuem prioridade alta;
- cobrança para cliente premium possui prioridade alta;
- cobrança para cliente comum possui prioridade média;
- demais casos possuem prioridade baixa;
- a ordem das regras influencia o resultado.

Testes Unitários Assistidos por IA

Gere testes unitários em Python usando pytest.
Crie um arquivo chamado `test_chamado.py`.

Requisitos:

- criar cenários positivos;
- validar diferentes fluxos condicionais;
- incluir edge cases;
- utilizar nomes descritivos para os testes;
- explicar brevemente quais regras foram testadas.

Não altere a função original.

10. Como avaliar os testes gerados pela IA

Depois que a IA gerar os arquivos de teste, revise se os cenários são relevantes e se realmente validam as regras de negócio. Um teste gerado automaticamente não deve ser aceito sem análise.

- **Cenários positivos:** confirmam que a função retorna o valor esperado para entradas válidas.
- **Cenários negativos:** verificam comportamentos inválidos, como entradas incorretas ou exceções esperadas.
- **Edge cases:** testam limites importantes, como exatamente 8 caracteres ou exatamente R\$ 300.
- **Nomes descritivos:** facilitam a leitura do relatório de falha quando o pytest encontra erro.
- **Assertivas objetivas:** cada teste deve validar um comportamento claro, sem excesso de lógica dentro do próprio teste.

11. Checklist de entrega

Antes de finalizar a atividade, confira se todos os itens abaixo foram atendidos.

- ☐ Foram criados arquivos `test_*.py` para os desafios realizados.
- ☐ Os testes foram executados com pytest.
- ☐ Há cenários positivos, negativos e pelo menos um edge case.
- ☐ As exceções esperadas foram validadas com `pytest.raises`, quando aplicável.
- ☐ Os nomes dos testes são claros e descrevem o comportamento validado.
- ☐ Os resultados da execução foram analisados.
- ☐ As falhas foram interpretadas tecnicamente, sem apenas copiar a resposta da IA.
- ☐ A função original não foi alterada durante a geração dos testes.

12. Resultado esperado da prática

Ao final da atividade, espera-se que você consiga transformar regras de negócio em testes automatizados, criar prompts mais completos para geração de testes, executar a suíte com pytest e interpretar falhas de forma técnica.

- Compreender o papel da IA como apoio, não como substituta da análise técnica.
- Gerar testes unitários mais consistentes usando contexto, regras e critérios claros.
- Identificar falsa cobertura, testes redundantes e casos irrelevantes.
- Explicar por que um teste falhou e qual regra de negócio foi afetada.

Fim do material